

1. Conéctese a AdventureWorksLT

Asegúrese de que la base de datos de muestra de AdventureWorksLT esté restaurada y disponible en su instancia SQL. Verificar la conectividad:

Copie y pegue el siguiente código T-SQL en una nueva ventana de consulta. Seleccione Ejecutar para ejecutar esta consulta:

```
-- Verify key tables in AdventureWorksLT
SELECT TOP (5) ProductID, Name, ListPrice
FROM SalesLT.Product;

SELECT TOP (5) ProductCategoryID, Name
FROM SalesLT.ProductCategory;
```

Cada consulta debe devolver hasta cinco filas de datos de muestra. Si alguna consulta no devuelve filas o falla, confirme que la base de datos AdventureWorksLT esté restaurada correctamente y que tenga acceso de lectura.

Resultados		Mensajes	
	ProductID	Name	ListPrice
1	680	HL Road Frame - Black, 58	1431,50
2	706	HL Road Frame - Red, 58	1431,50
3	707	Sport-100 Helmet, Red	34,99
4	708	Sport-100 Helmet, Black	34,99
5	709	Mountain Bike Socks, M	9,50

	ProductCategoryID	Name
1	4	Accessories
2	22	Bib-Shorts
3	30	Bike Racks
4	31	Bike Stands
5	1	Bikes

2. Generar salida JSON a partir de datos del producto

El equipo de marketing necesita información del producto en formato JSON para un catálogo web. Comience creando un objeto JSON simple a partir de la tabla Producto.

2.1. Crea un objeto JSON para cada producto

Úselo **FOR JSON PATH** para convertir filas de productos a JSON.

Copie y pegue el siguiente código T-SQL en una nueva ventana de consulta. Seleccione Ejecutar para ejecutar esta consulta:

SQL

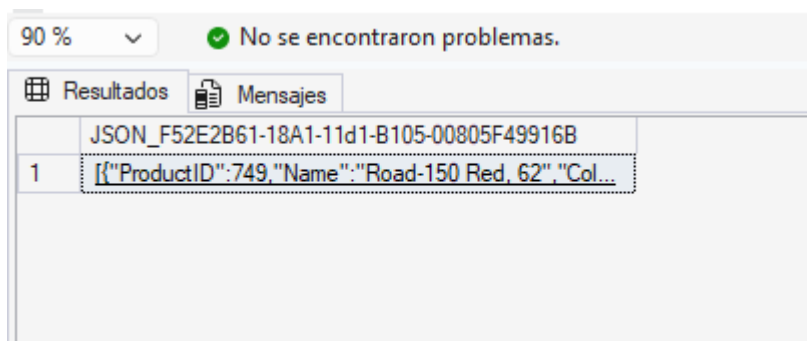
```
-- =====
-- 2.1. Generación de un array JSON plano a partir de la tabla Product
-- =====
SELECT
    ProductID,
    Name,
```

```

Color,
ListPrice
FROM SalesLT.Product
WHERE Color IS NOT NULL
ORDER BY ListPrice DESC
FOR JSON PATH;

```

Esta consulta selecciona productos con un valor de color y formatea los resultados como una matriz JSON. Cada fila se convierte en un objeto JSON con propiedades que coinciden con los nombres de las columnas. La cláusula **FOR JSON PATH** maneja la conversión automáticamente.



2.2. Crear JSON anidado con categorías de productos

Agregue información de categoría como un objeto anidado.

Copie y pegue el siguiente código T-SQL en una nueva ventana de consulta. Seleccione Ejecutar para ejecutar esta consulta:

SQL

```

-- =====
-- 2.2. Construcción de objetos JSON anidados con JSON_OBJECT()
-- =====

SELECT
    p.ProductID,
    p.Name AS ProductName,
    p.ListPrice,
    JSON_OBJECT(
        'CategoryID': pc.ProductCategoryID,
        'CategoryName': pc.Name
    ) AS Category
FROM SalesLT.Product AS p
INNER JOIN SalesLT.ProductCategory AS pc
    ON p.ProductCategoryID = pc.ProductCategoryID
ORDER BY p.ListPrice DESC
FOR JSON PATH;

```

Esta consulta utiliza **JSON_OBJECT** para construir una estructura anidada. La propiedad Categoría contiene su propio objeto JSON con **CategoryID** y **CategoryName**. Este enfoque mantiene los datos relacionados agrupados en el resultado.

90 % ✓ No se encontraron problemas.

Resultados		Mensajes
	JSON_F52E2B61-18A1-11d1-B105-00805F49916B	
1	[{"ProductID":749,"ProductName":"Road-150 Red, 6...	

3. Combine JSON con una función CTE y de ventana

Ahora cree un informe más útil que clasifique los productos por precio dentro de cada categoría y genere el resultado como JSON.

3.1. Escriba un CTE con clasificación de funciones de ventana

Primero, construya la lógica de consulta usando un CTE y **ROW_NUMBER()**.

Copie y pegue el siguiente código T-SQL en una nueva ventana de consulta. Seleccione Ejecutar para ejecutar esta consulta:

SQL

```
-- =====
-- 3.1. Ranking por partición con Common Table Expression (CTE) y ROW_NUMBER()
WITH RankedProducts AS (
    SELECT
        p.ProductID,
        p.Name AS ProductName,
        pc.Name AS CategoryName,
        p.ListPrice,
        ROW_NUMBER() OVER (
            PARTITION BY pc.ProductCategoryID
            ORDER BY p.ListPrice DESC
        ) AS PriceRank
    FROM SalesLT.Product AS p
    INNER JOIN SalesLT.ProductCategory AS pc
        ON p.ProductCategoryID = pc.ProductCategoryID
    WHERE p.ListPrice > 0
)
SELECT
    ProductID,
    ProductName,
```

```

    CategoryName,
    ListPrice,
    PriceRank
FROM RankedProducts
WHERE PriceRank <= 3
ORDER BY CategoryName, PriceRank;

```

El CTE calcula un rango de precios para cada producto dentro de su categoría. La cláusula **PARTITION BY** reinicia la numeración de cada categoría y **ORDER BY ListPrice DESC** asigna el rango 1 al producto más caro. Los filtros de consulta externos muestran solo los 3 productos principales por categoría.

90 % ✔ No se encontraron problemas.

Resultados Mensajes

	ProductID	ProductName	CategoryName	ListPrice	PriceRank
1	855	Men's Bib-Shorts, S	Bib-Shorts	89,99	1
2	856	Men's Bib-Shorts, M	Bib-Shorts	89,99	2
3	857	Men's Bib-Shorts, L	Bib-Shorts	89,99	3
4	876	Hitch Rack - 4-Bike	Bike Racks	120,00	1
5	879	All-Purpose Bike Stand	Bike Stands	159,00	1
6	871	Mountain Bottle Cage	Bottles and Cages	9,99	1
7	872	Road Bottle Cage	Bottles and Cages	8,99	2
8	870	Water Bottle - 30 oz.	Bottles and Cages	4,99	3
9	996	HL Bottom Bracket	Bottom Brackets	121,49	1
10	995	ML Bottom Bracket	Bottom Brackets	101,24	2
11	994	LL Bottom Bracket	Bottom Brackets	53,99	3
12	948	Front Brakes	Brakes	106,50	1
13	907	Rear Brakes	Brakes	106,50	2
14	712	AWC Logo Cap	Caps	8,99	1
15	952	Chain	Chains	20,24	1
16	877	Bike Wash - Dissolver	Cleaners	7,95	1
17	951	HL Crankset	Cranksets	404,99	1
18	950	ML Crankset	Cranksets	256,49	2
19	949	LL Crankset	Cranksets	175,49	3
20	894	Rear Derailleur	Derailleurs	121,46	1
21	945	Front Derailleur	Derailleurs	91,49	2
22	878	Fender Set - Mountain	Fenders	21,98	1
23	804	HL Fork	Forks	229,49	1
24	803	ML Fork	Forks	175,49	2
25	802	LL Fork	Forks	148,22	3
26	861	Full-Finger Gloves, S	Gloves	37,99	1
27	862	Full-Finger Gloves, M	Gloves	37,99	2
28	863	Full-Finger Gloves, L	Gloves	37,99	3
29	810	HL Mountain Handlebars	Handlebars	120,27	1
30	811	ML Mountain Handlebars	Handlebars	100,27	2
31	812	LL Mountain Handlebars	Handlebars	80,27	3

✔ Consulta ejecutada correctamente.

3.2. Genere los productos clasificados como JSON

Agregar **FOR JSON PATH** para formatear los resultados de una API.

Copie y pegue el siguiente código T-SQL en una nueva ventana de consulta. Seleccione Ejecutar para ejecutar esta consulta:

SQL

```
-- =====
-- 3.2. Exportar el Top 3 clasificado a JSON con nodo raíz ROOT()
-- =====

WITH RankedProducts AS (
    SELECT
        p.ProductID,
        p.Name AS ProductName,
        pc.Name AS CategoryName,
        p.ListPrice,
        ROW_NUMBER() OVER (
            PARTITION BY pc.ProductCategoryID
            ORDER BY p.ListPrice DESC
        ) AS PriceRank
    FROM SalesLT.Product AS p
    INNER JOIN SalesLT.ProductCategory AS pc
        ON p.ProductCategoryID = pc.ProductCategoryID
    WHERE p.ListPrice > 0
)
SELECT
    ProductID,
    ProductName,
    CategoryName,
    ListPrice,
    PriceRank
FROM RankedProducts
WHERE PriceRank <= 3
ORDER BY CategoryName, PriceRank
FOR JSON PATH, ROOT('TopProducts');
```

La adición **ROOT('TopProducts')** envuelve toda la matriz JSON en un objeto con una propiedad con nombre. Esto hace que sea más fácil trabajar con el resultado en aplicaciones que esperan un elemento raíz.

JSON_F52E2B...9916B2.json JSON_F52E2B6...49916B1.json SQLQuery1.sql

```
1  {
2    "TopProducts": [
3      {
4        "ProductID": 855,
5        "ProductName": "Men's Bib-Shorts, S",
6        "CategoryName": "Bib-Shorts",
7        "ListPrice": 89.9900,
8        "PriceRank": 1
9      },
10     {
11       "ProductID": 856,
12       "ProductName": "Men's Bib-Shorts, M",
13       "CategoryName": "Bib-Shorts",
14       "ListPrice": 89.9900,
15       "PriceRank": 2
16     },
17     {
18       "ProductID": 857,
19       "ProductName": "Men's Bib-Shorts, L",
20       "CategoryName": "Bib-Shorts",
21       "ListPrice": 89.9900,
22       "PriceRank": 3
23     },
24     {
25       "ProductID": 876,
26       "ProductName": "Hitch Rack - 4-Bike",
27       "CategoryName": "Bike Racks",
28       "ListPrice": 120.0000,
29       "PriceRank": 1
30     },
31     {
32       "ProductID": 879,
33       "ProductName": "All-Purpose Bike Stand",
34       "CategoryName": "Bike Stands",
35       "ListPrice": 159.0000,
36       "PriceRank": 1
37     },
38     {
39       "ProductID": 871,
40       "ProductName": "Mountain Bottle Cage",
41       "CategoryName": "Bottles and Cages",
42       "ListPrice": 9.9900,
43       "PriceRank": 1
44     },
45     {
46       "ProductID": 872,
47       "ProductName": "Road Bottle Cage",
48       "CategoryName": "Bottles and Cages",
49       "ListPrice": 8.9900,
50       "PriceRank": 2
51     },
52     {
53       "ProductID": 870,
54       "ProductName": "Water Bottle - 30 oz "
```

90 % No se encontraron problemas.

4. Analizar datos JSON con OPENJSON

Ahora practique la lectura de datos JSON nuevamente en filas usando **OPENJSON**.

4.1. Analizar una matriz JSON en filas

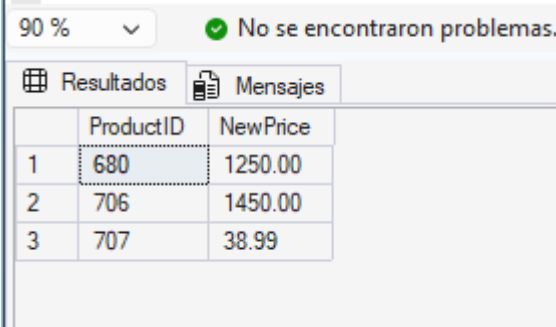
Supongamos que recibe actualizaciones de productos como JSON. Úselo **OPENJSON** para convertirlo en una tabla.

Copie y pegue el siguiente código T-SQL en una nueva ventana de consulta. Seleccione Ejecutar para ejecutar esta consulta:

SQL

```
-- =====  
-- 4.1. Deserializar un string JSON en filas tabulares usando OPENJSON y WITH  
-- =====  
-- Variable que simula un payload JSON entrante (ej. desde una API o webhook)  
DECLARE @ProductUpdates NVARCHAR(MAX) = N'  
    {"ProductID": 680, "NewPrice": 1250.00},  
    {"ProductID": 706, "NewPrice": 1450.00},  
    {"ProductID": 707, "NewPrice": 38.99}  
';  
  
SELECT  
    ProductID,  
    NewPrice  
FROM OPENJSON(@ProductUpdates)  
WITH (  
    ProductID INT '$.ProductID',  
    NewPrice DECIMAL(10,2) '$.NewPrice'  
) ;
```

La cláusula **WITH** define el esquema para la salida. Cada propiedad JSON se asigna a una columna con un tipo de datos específico. La sintaxis **\$.PropertyName** le dice a SQL Server qué ruta JSON leer para cada columna.



90 % No se encontraron problemas.

	ProductID	NewPrice
1	680	1250.00
2	706	1450.00
3	707	38.99

4.2. Unir JSON analizado con datos existentes

Combine los datos JSON con la tabla Producto para ver los precios actuales y nuevos.

Copie y pegue el siguiente código T-SQL en una nueva ventana de consulta. Seleccione Ejecutar para ejecutar esta consulta:

SQL

```
-- =====
-- 4.2. JOIN entre una tabla relacional existente y datos procesados con OPENJSON
-- =====

DECLARE @ProductUpdates NVARCHAR(MAX) = N'[
  {"ProductID": 680, "NewPrice": 1250.00},
  {"ProductID": 706, "NewPrice": 1450.00},
  {"ProductID": 707, "NewPrice": 38.99}
]';

SELECT
  p.ProductID,
  p.Name,
  p.ListPrice AS CurrentPrice,
  updates.NewPrice,
  updates.NewPrice - p.ListPrice AS PriceDifference
FROM SalesLT.Product AS p
INNER JOIN OPENJSON(@ProductUpdates)
WITH (
  ProductID INT '$.ProductID',
  NewPrice DECIMAL(10,2) '$.NewPrice'
) AS updates
ON p.ProductID = updates.ProductID;
```

Esta consulta une el JSON analizado directamente con la tabla Producto. La función **OPENJSON** con una cláusula **WITH** actúa como una tabla, por lo que puedes unirla como cualquier otra fuente de datos. El resultado muestra el precio actual de cada producto junto con el nuevo precio propuesto.

90 % No se encontraron problemas.

Resultados Mensajes

	ProductID	Name	CurrentPrice	NewPrice	PriceDifference
1	680	HL Road Frame - Black, 58	1431,50	1250.00	-181.5000
2	706	HL Road Frame - Red, 58	1431,50	1450.00	18.5000
3	707	Sport-100 Helmet, Red	34,99	38.99	4.0000